

# **Local Outlier Factor (LOF)**

Détection d'Anomalies basée sur la Densité Locale

Cours de Machine Learning Non Supervisé

28 décembre 2025

## **Table des matières**

# 1 Introduction

Le **Local Outlier Factor** (LOF) (Breunig et al., 2000) est une méthode pionnière de détection d'anomalies. Contrairement aux méthodes globales (qui comparent un point à tous les autres), LOF s'intéresse à la **densité locale**.

## Concept Clé

### Intuition Fondamentale :

Une anomalie est un point qui est **plus isolé** que ses voisins.

- Si un point a une densité similaire à celle de ses voisins, c'est un point normal.
- Si sa densité est **beaucoup plus faible** que celle de ses voisins, c'est une anomalie.

L'avantage majeur est de pouvoir détecter des anomalies dans des zones de densités différentes, là où un seuil de distance global échouerait.

## 2 Principe de Fonctionnement

LOF calcule un score pour chaque point, basé sur le rapport entre sa densité locale et la densité locale moyenne de ses  $k$  plus proches voisins.

### 2.1 Illustration

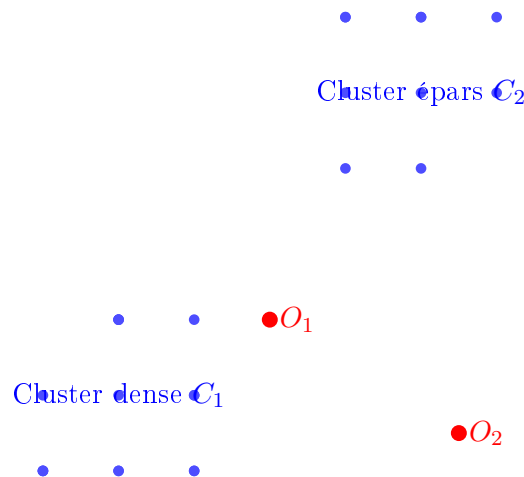


Figure 1 : LOF permet de détecter  $O_1$  comme anomalie relative à  $C_1$ , même si la distance entre points dans  $C_2$  est similaire.

## 3 Fondements Mathématiques

L'algorithme se déroule en 4 étapes clés :

### 3.1 1. k-Distance et Voisins

Pour un point  $p$  et un entier  $k$ , la  $k$ -distance( $p$ ) est la distance au  $k$ -ième voisin le plus proche.  $N_k(p)$  est l'ensemble de ces  $k$  voisins (peut être  $> k$  en cas d'égalité).

### 3.2 2. Reachability Distance (Distance d'Atteignabilité)

C'est une distance "lissée". Pour éviter les problèmes si des points sont superposés :

$$\text{reach-dist}_k(p, o) = \max\{k\text{-distance}(o), d(p, o)\}$$

où  $o \in N_k(p)$ . Intuition : Si  $p$  est très proche de  $o$ , on utilise la  $k$ -distance de  $o$  comme "bulle de protection" minimale.

### 3.3 3. Local Reachability Density (LRD - Densité Locale)

La densité locale de  $p$  est l'inverse de la distance d'atteignabilité moyenne de  $p$  par rapport à ses voisins :

$$\text{lrd}_k(p) = \frac{|N_k(p)|}{\sum_{o \in N_k(p)} \text{reach-dist}_k(p, o)}$$

### 3.4 4. Local Outlier Factor (LOF)

Le score final compare la LRD de  $p$  à celles de ses voisins  $o$  :

$$\text{LOF}_k(p) = \frac{\sum_{o \in N_k(p)} \frac{\text{lrd}_k(o)}{\text{lrd}_k(p)}}{|N_k(p)|} = \frac{\text{Moyenne LRD voisins}}{\text{LRD du point}}$$

#### Fondement Mathématique

##### Interprétation du Score LOF

- $\text{LOF} \approx 1$  : Densité similaire aux voisins  $\Rightarrow$  **Normal**.
- $\text{LOF} \ll 1$  : Point plus dense que ses voisins  $\Rightarrow$  **Intérieur de cluster**.
- $\text{LOF} \gg 1$  : Densité beaucoup plus faible que les voisins  $\Rightarrow$  **Anomalie**.

En pratique, un seuil  $> 1.5$  ou  $> 2.0$  est souvent utilisé pour flagger une anomalie.

## 4 Hyperparamètre Principal : n\_neighbors

Le seul paramètre critique est  $k$  (`n_neighbors` dans Scikit-Learn).

- $k$  trop petit (ex : 1 ou 2) : Sensible au bruit, faux positifs locaux.
- $k$  trop grand : L'effet local se dilue, on tend vers une approche globale.
- **Recommandation** : Souvent  $k = 20$  par défaut. Breunig suggère une plage (`minPts`, `maxPts`).

## 5 Comparaison avec Isolation Forest et SVM

| Méthode          | Force                            | Faiblesse                            |
|------------------|----------------------------------|--------------------------------------|
| Isolation Forest | Rapide, Scalable, Global         | Rate les anomalies locales denses    |
| One-Class SVM    | Frontières complexes             | Lent, Paramétrage difficile          |
| <b>LOF</b>       | <b>Détecte anomalies locales</b> | Lent ( $O(n^2)$ ), Pas de prédiction |

#### Attention

**Nouveauté et Prédiction** : Standard LOF est conçu pour la détection d'outliers dans un dataset statique (*transductif*). Il ne prédit pas nativement sur de nouvelles données sans recalculer les distances.  $\Rightarrow$  Scikit-Learn propose `novelty=True` pour l'utiliser comme un estimateur qui peut appeler `predict()`, mais attention : cela change légèrement la logique.

## 6 Exemple de Code Python

### Implémentation Python

```
1 from sklearn.neighbors import LocalOutlierFactor
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.metrics import classification_report, confusion_matrix
4 import numpy as np
5
6 # 1. PR PARATION
7 # Les m thodes de distance n cessitent une normalisation !
8 scaler = StandardScaler()
9 X_scaled = scaler.fit_transform(X)
10
11 # 2. MOD LISATION
12 # n_neighbors=20 : standard
13 # contamination=0.03 : on s'attend 3% d'anomalies
14 # novelty=False : On detecte sur le jeu d'entrainement lui-m me (
    classique)
15 lof = LocalOutlierFactor(
16     n_neighbors=20,
17     contamination=0.03,
18     n_jobs=-1
19 )
20
21 print("Calcul du LOF...")
22 # fit_predict existe pour le mode outlier detection
23 preds = lof.fit_predict(X_scaled)
24
25 # 3. CONVERSION
26 # LOF : 1 = Normal, -1 = Anomalie
27 y_pred_lof = np.where(preds == -1, 1, 0)
28
29 # Extraire les scores bruts (negative_outlier_factor_)
30 # Plus c'est negatif, plus c'est une anomalie (dans sklearn
    implementation)
31 lof_scores = -lof.negative_outlier_factor_
32
33 # 4. VALUATION
34 print("Matrice de Confusion :")
35 print(confusion_matrix(y, y_pred_lof))
36
37 print("\nRapport :")
38 print(classification_report(y, y_pred_lof, target_names=['Normal', 'Panne'
    ]))
39
40 # Note : lof_scores > 1.5 indiquent g n rarement une anomalie
41 print(f"Score LOF max : {lof_scores.max():.2f}")
42 print(f"Score LOF moyen : {lof_scores.mean():.2f}")
```

## 7 Conclusion

LOF est l'outil de choix lorsque les anomalies sont contextuelles (ex : une température de 20°C est normale en été, mais anormale en hiver). Sa capacité à s'adapter à la densité locale le rend supérieur aux approches globales dans des datasets hétérogènes, au prix d'un coût de calcul plus élevé.